

Modules 7 & 8 — Question Pool

OMSCS 6250 Computer Networks

Lesson 7 + Lesson 8: Software Defined Networking (SDN)

What led us to SDN?

Q1. [MCQ]

Why are traditional networks hard to manage, in the view this lesson presents?

- **A.** The diversity of devices (routers, switches, middleboxes) and the proprietary, vendor-specific control software running on each makes consistent network-wide policy hard to express and verify.
- **B.** Modern devices are too slow at line rate, so operators must continually replace hardware to keep up.
- **C.** IPv6 transition fundamentally broke the management plane that worked under IPv4.
- **D.** Open-source software dominates the data plane and lacks the maturity of vendor-shipped systems.

Correct answer: **A**

Why: Vendor heterogeneity + proprietary control SW => inconsistent policy. Each box runs its own CLI, control software, and configuration model; expressing 'route X via path P everywhere, enforce Y everywhere' becomes a per-device puzzle rather than a global statement.

Q2. [TF]

In traditional networks, the control logic and the packet-forwarding logic are tightly integrated inside the same physical box.

- True
- False

Correct answer: **True**

Why: Same box = same CPU runs both planes. In traditional routers, the route processor and forwarding ASICs sit inside one physical chassis; the conceptual split exists but they're not separable.

A Brief History of SDN: The Milestones

Q3. [MCQ]

The history of SDN is often described in three phases. Which list is correct in chronological order?

- **A.** ARPANET; ATM; MPLS — these are the three precursors of modern SDN.

- **B.** Active networks; control/data plane separation; OpenFlow and network operating systems.
- **C.** Active networks; MPLS; P4 hardware programmability.
- **D.** OpenFlow; ONOS; SD-WAN deployment in enterprises.

Correct answer: **B**

Why: Active networks -> control/data separation -> OpenFlow + NOSes. Active networks (1990s) explored programmable packets; the early 2000s formalized control-plane decoupling (4D, RCP); OpenFlow + NOX/ONIX/ONOS made it deployable.

Q4. [TF]

The conceptual separation of the control plane from the data plane PREDATES OpenFlow by several years, even though OpenFlow is what made the separation practical.

- True
- False

Correct answer: **True**

Why: Separation idea predates OpenFlow by years. RCP (2005) and the 4D project argued for centralised control before OpenFlow (2008) shipped a concrete southbound API that made it practical.

Why Separate the Data Plane from the Control Plane?

Q5. [MCQ]

The lesson argues that separating control and data planes enables several benefits at once. Which one is BEST captured by 'a logically centralized view of the network'?

- **A.** Forwarding decisions are pushed up to the end hosts directly, while the switches are no longer responsible for making forwarding choices.
- **B.** Each device runs its own routing algorithm independently, exactly as in traditional networks, but with cleaner code.
- **C.** The controller has the whole topology in one place, so it can compute consistent policies (routing, security, traffic engineering) across many devices.
- **D.** The data plane gains write access to the global routing table for every other AS.

Correct answer: **C**

Why: Centralized view => consistent global policy. The controller sees the entire topology and computes coherent forwarding/security/TE decisions; per-device protocols can only converge to consistency, not guarantee it.

Q6. [MCQ]

In a traditional network, changing a routing or security policy across many devices requires logging into each device's CLI and applying per-device commands. In SDN, the same change is expressed once at the

controller. Beyond saving keystrokes, which OPERATIONAL property does this centralization enable that the traditional CLI workflow makes hard?

- **A.** Routers no longer need a control plane — the SDN controller forwards every packet itself, removing the need for operator oversight.
- **B.** Hardware-level encryption is automatically enforced on every flow without any operator action or configuration.
- **C.** The data plane no longer needs forwarding logic; the controller is consulted over the network for each packet before the packet is forwarded.
- **D.** Network configuration becomes centralized, software-managed state — so changes can be version-controlled (e.g., Git), code-reviewed, tested in CI, and rolled back like any other software artifact.

Correct answer: **D**

Why: Centralized config = code-managed state => Git, code review, CI, rollback. Per-device CLIs scatter config across boxes with no native version control or atomic rollback; once the controller holds the network's intent as software-defined state, every change flows through the same pipelines (PR, test, deploy, revert) that software teams already use — eliminating a major class of operational risk.

Control Plane and Data Plane Separation

Q7. [MCQ]

Consider an SDN network, where does the routing computation occur?

- **A.** On a logically centralized controller that distributes forwarding tables to the network elements.
- **B.** Inside each switch, identically to the traditional approach but with newer firmware.
- **C.** On end hosts that originate the traffic and intermediate switches/routers as Layer-1 repeaters.
- **D.** SDN networks include IXP route servers which are configured to perform routing.

Correct answer: **A**

Why: Routing computed on centralized controller. The controller runs algorithms on the global view and pushes resulting forwarding rules down to the switches via the southbound API.

Q8. [MCQ]

Consider an SDN run network where the SDN controller runs in a remote data center which is not co-located with the routers. Which architectural design/property of SDN run networks enables this operation?

- **A.** The data plane includes a built-in tunnel to the operator's headquarters for any remote management traffic.
- **B.** The control plane is implemented in software and is physically decoupled from the data plane — it can run anywhere reachable over a control channel.
- **C.** Routers run the full controller code locally and send commands to themselves via loopback.

- **D.** OpenFlow requires the controller to be co-located, but operators may pre-stage forwarding rules from afar.

Correct answer: **B**

Why: Control plane is SW + physically decoupled => runs anywhere reachable. Because the control channel is just network traffic, the controller can sit in a different rack, data center, or even cloud — wherever the operator wants.

Q9. [TF]

In traditional (non-SDN) networks, separating data plane and control plane CONCEPTUALLY already existed — but they ran inside the same physical box, on the same CPU.

- True
- False

Correct answer: **True**

Why: Conceptual separation existed, but co-located on same CPU. Even traditional routers distinguish 'compute routes' from 'forward packets'; SDN's contribution is making the separation physical and programmable.

The SDN Architecture

Q10. [MCQ]

In the SDN-architecture figure, the 'infrastructure layer' (the SDN-controlled switches) is responsible for which function?

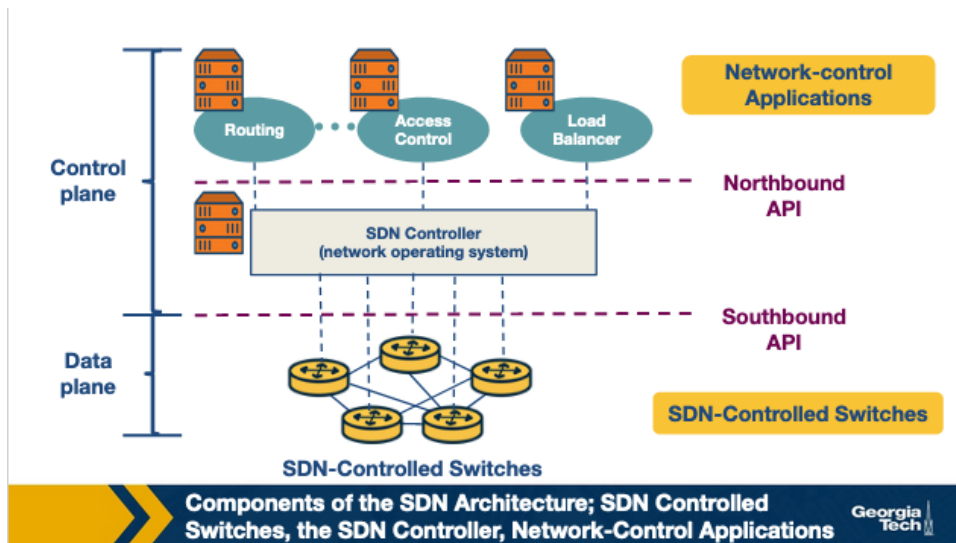
- **A.** Acting as the northbound REST API that operator tools talk to.
- **B.** Computing forwarding rules based on operator policy directly within the switch CPU.
- **C.** Hosting the network-control applications that operators write for the controller.
- **D.** Forwarding traffic according to the rules computed and installed by the SDN control plane above.

Correct answer: **D**

Why: Infrastructure layer = switches that execute installed rules. The data-plane elements (OpenFlow switches, P4 targets) accept rules from the controller and forward at line rate; they don't compute routes themselves.

Q11. [MCQ]

Which SDN interface connects the CONTROLLER to the network-control applications above (e.g., a REST API)?



Reference Figure from Canvas Module: SDN architecture (Module 7)

- A. Southbound interface (e.g., OpenFlow).
- B. Northbound interface.
- C. Mininet topology interface.
- D. OpenFlow flow-table priority field.

Correct answer: **B**

Why: Northbound = controller <-> apps (REST API). Operator-written control applications talk to the controller through the northbound interface to express policy and read network state.

Q12. [MCQ]

Which SDN interface connects the CONTROLLER to the switches it programs (e.g., OpenFlow)?

- A. East-west interface.
- B. Northbound interface.
- C. Southbound interface.
- D. Management plane interface.

Correct answer: **C**

Why: Southbound = controller <-> switches (OpenFlow). The southbound API installs/removes flow rules in the switches and receives switch events (port up/down, packet-in) — OpenFlow is the canonical example.

Q13. [TF]

In the SDN architecture, the controller is responsible for COMPUTING forwarding decisions; the switches are responsible for EXECUTING those forwarding decisions at line rate.

- True
- False

Correct answer: **True**

Why: Controller computes; switches execute. Cleanly split responsibility: route decisions and policy live in software at the controller; per-packet enforcement at line rate happens on the switches.

The SDN Controller Architecture

Q14. [MCQ]

Modern SDN controllers are typically DISTRIBUTED across multiple instances. Select the primary motivation.

- **A.** Eliminating the southbound API entirely, since distributed controllers talk to switches via gossip.
- **B.** Reducing latency below 1 nanosecond per control message.
- **C.** Allowing controllers to be written in many different programming languages.
- **D.** Fault tolerance and scalability — a single instance is a single point of failure and may not handle the load of a large network.

Correct answer: **D**

Why: Distributed controllers = fault tolerance + scale. A single controller is a SPOF and a bottleneck; distributing to multiple instances tolerates failures and shares load across a large network.

Q15. [TF]

Logical centralization of the control plane does NOT require physical centralization — a distributed control platform can still present a single logical view to applications.

- True
- False

Correct answer: **True**

Why: Logical != physical centralization. The control platform can be distributed (for HA and scale) yet still present applications with a single logical view of the network through shared state.

Revisiting the Motivation for SDN

Q16. [MCQ]

Consider the differences between traditional networks and SDN run networks. Which architectural change one of the most fundamental that SDN introduces?

- **A.** Separating the control logic from the data plane and placing the logic in a logically centralized controller.
- **B.** Replacing IP with a new Layer-3 protocol designed specifically for software-defined switches.

- C. Eliminating the data plane entirely so the controller forwards every packet itself.
- D. Forcing every switch to run BGP as its intra-AS routing protocol.

Correct answer: **A**

Why: Separation of control logic + centralized controller. The defining architectural shift: control reasoning moves out of each box and into a logical controller that programs the data plane via a southbound API.

Q17. [TF]

SDN does not eliminate the data plane — the data plane is still where packets are actually forwarded at line rate. SDN only moves the CONTROL logic out of the device.

- True
- False

Correct answer: **True**

Why: SDN doesn't kill the data plane — it moves CONTROL. Packets still get forwarded at line rate by the switch ASICs; what's been factored out is the route-computation logic, not the forwarding itself.

SDN Advantages

Q18. [MCQ]

How does SDN benefit middlebox-like services in practice?

- A. Every middlebox in the data plane is physically replaced by a local copy of the SDN controller, so each box independently recomputes the global forwarding policy.
- B. A single common control platform can express firewall, NAT, load-balancing, and other policies without each one being a separate proprietary middlebox.
- C. Operators stop using firewalls, NAT, and load balancers entirely once they deploy SDN.
- D. Middleboxes still exist but no longer participate in the data path at all.

Correct answer: **B**

Why: One control platform expresses many policies (firewall, NAT, LB). Instead of buying a separate proprietary middlebox per function, SDN apps express those policies on top of a common controller and switch fabric.

Q19. [MCQ — select all that apply]

The lesson presents SDN by contrasting it with conventional networks built from proprietary middleboxes. Which of the following are genuine advantages of the SDN approach as described in the lesson?

- A. A common control platform can express firewall, NAT, and load-balancing policies as applications, instead of a separate proprietary middlebox for each function.

- **B.** Deploying SDN removes the need for firewalls, NAT, and load balancers.
- **C.** Every network application works from the same global view of the network, keeping policy decisions consistent and letting control-plane modules be reused.
- **D.** A network function's logic can run from anywhere in the network, rather than being pinned to a fixed physical location the way middleboxes were.
- **E.** Each switch hosts its own copy of the controller and recomputes the global forwarding policy on its own.
- **F.** Independent applications, such as routing and load balancing, can be combined and integrated more smoothly.
- **G.** The control and data planes stay bundled inside each device, now running on faster hardware.
- **H.** Decoupling the control plane from the data plane lets new features and protocols roll out without upgrading firmware on every device.

Correct answer: **A, C, D, F, H**

Why: A / C / D / F are the four advantages the "SDN Advantages" page names — shared abstractions, consistent global network view, location-independent placement, and simpler integration. H is the core motivation benefit: decoupling the planes ends the per-device, ~10-year rollout cycle. B is the classic trap — those functions aren't removed; they become applications on the controller. E is wrong — control is logically centralized (a controller, physically distributed in practice), not replicated per switch. G is wrong — SDN's whole premise is decoupling the control and data planes, not keeping them bundled.

The SDN Landscape

Q20. [MCQ]

The figure shows three different ways to view the SDN landscape. Which set names the three views correctly?

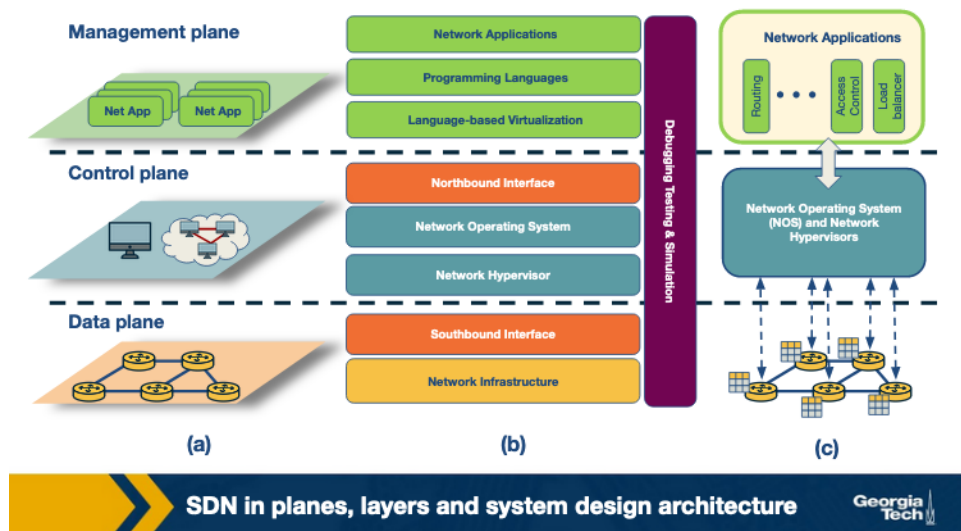


Figure: SDN landscape — plane-oriented, layered, and system-design views (Module 8)

- **A.** The OSI seven-layer model, the TCP/IP four-layer model, and the legacy ATM cell-switching reference model.
- **B.** Physical layout, IP addressing, and routing tables.

- C. BGP topology, DNS hierarchy, and TLS handshake.
- D. A plane-oriented view, a layers view (infrastructure/control/application), and a system-design perspective.

Correct answer: **D**

Why: Three views: plane-oriented, layered, system-design. The lesson decomposes the SDN landscape from three angles — control/data planes, infra/control/application layers, and the broader system-design considerations.

SDN Infrastructure Layer

Q21. [MCQ]

In the OpenFlow-derived SDN data plane, each flow-table entry has three parts. Which set is correct?

- A. A source MAC address, a destination MAC address, and a VLAN tag used as the entry's match key.
- B. A matching rule, actions to execute on matching packets, and counters for statistics.
- C. A timeout, a priority, and an encryption key.
- D. A parser, a deparser, and a control program.

Correct answer: **B**

Why: Match + actions + counters. An OpenFlow flow entry is (matching rule, action list, counters) — counters give per-flow stats, actions specify forward/drop/modify, and the rule defines the header pattern.

Q22. [TF]

A single OpenFlow device can have multiple flow tables forming a processing PIPELINE, with each table either matching a packet or passing it onward.

- True
- False

Correct answer: **True**

Why: Multi-table pipelines forward through stages. Modern OpenFlow exposes multiple flow tables; each stage matches/acts and can pass packets to the next table — enabling clean separation of L2/L3/ACL logic.

SDN Southbound Interfaces

Q23. [MCQ]

OpenFlow defines three categories of information that forwarding devices send to the controller. Which is NOT one of them?

- **A.** Packet messages (PacketIn — when a device does not know what to do with a flow).
- **B.** Event-based messages (e.g., link/port state change).
- **C.** Flow statistics (counters per flow).
- **D.** BGP update messages exchanged between border routers.

Correct answer: **D**

Why: BGP UPDATE is NOT one of OpenFlow's three info sources. OpenFlow's switch-to-controller messages are PacketIn (unknown flow), events (link/port up/down), and statistics (per-flow counters).

Q24. [MCQ]

Which statement BEST characterizes OpenFlow's role in the SDN architecture, as described in the module?

- **A.** OpenFlow is a southbound API/protocol used by the controller to program the data plane — it is the interface between the control and data planes.
- **B.** OpenFlow is a full control plane that runs locally on each switch and replaces the external controller, computing all routing decisions on the switch itself.
- **C.** OpenFlow is a forwarding fabric used in place of Ethernet at the data plane.
- **D.** OpenFlow is a management-plane SNMP replacement for legacy network monitoring.

Correct answer: **A**

Why: OpenFlow = southbound protocol (controller <-> switch). It's the API by which the controller programs flow tables in the data plane — neither a forwarding fabric nor a control plane on the switch itself.

Q25. [MCQ]

Consider an SDN application that wants to continuously monitor link utilization for QoS adjustments. Which of OpenFlow's three info sources fits BEST, and why?

- **A.** PacketIn — every monitored packet must be sent up to the controller so utilization can be measured.
- **B.** Flow statistics — they continuously expose per-flow byte/packet counters that the controller can read on demand without per-packet involvement.
- **C.** Event-based messages — link utilization is computed only when a link goes up or down.
- **D.** BGP UPDATE messages — utilization is encoded in BGP UPDATE counters by convention.

Correct answer: **B**

Why: Flow statistics => per-flow counters, polled on demand. PacketIn would generate too much controller traffic for utilization monitoring; statistics let the controller read byte/packet counts without touching every packet.

Q26. [MCQ]

Consider a scenario where a packet arrives at an OpenFlow switch and no flow-table entry matches it. Which of these BEHAVIORS is consistent with what the lesson describes?

- **A.** The switch must always forward to the controller; dropping on miss is never permitted by the spec.
- **B.** The switch must always drop the packet immediately, with no exception.
- **C.** Either dropping the packet OR encapsulating it and forwarding to the controller — both are valid outcomes depending on configured actions.
- **D.** The switch must broadcast the packet on every port to discover routes.

Correct answer: **C**

Why: Miss policy: drop OR PacketIn to controller (per config). The action on a table-miss is configurable; common defaults send the packet to the controller (slow-path discovery) or just drop it.

SDN Controllers: Centralized vs Distributed

Q27. [MCQ]

A centralized SDN controller architecture has what main characteristic, as described in the module?

- **A.** It uses BGP as the southbound API by default.
- **B.** Multiple controllers vote on every packet to decide each forwarding rule democratically, and a majority quorum must agree before any rule is installed.
- **C.** It eliminates the southbound interface by pushing all logic into switch hardware.
- **D.** A single entity manages all forwarding devices — simple in design, but a single point of failure and potentially a scaling bottleneck.

Correct answer: **D**

Why: Centralized = one entity manages everything (simple but SPOF). Easiest architecture to reason about, but the single controller is a single point of failure and a scaling ceiling.

Q28. [MCQ]

Which two properties are emphasized for DISTRIBUTED SDN controllers?

- **A.** Weak consistency semantics across instances and fault tolerance.
- **B.** Strict serializability and zero-copy I/O.
- **C.** Hardware encryption and BGP federation.
- **D.** VLAN tagging and MPLS labeling support.

Correct answer: **A**

Why: Distributed controllers => weak consistency + fault tolerance. Spreading control across instances trades strict global consistency for availability and load distribution; the controller designer picks the consistency model.

An Example Controller: ONOS

Q29. [MCQ]

In the ONOS figure, how is each switch associated with controller instances?

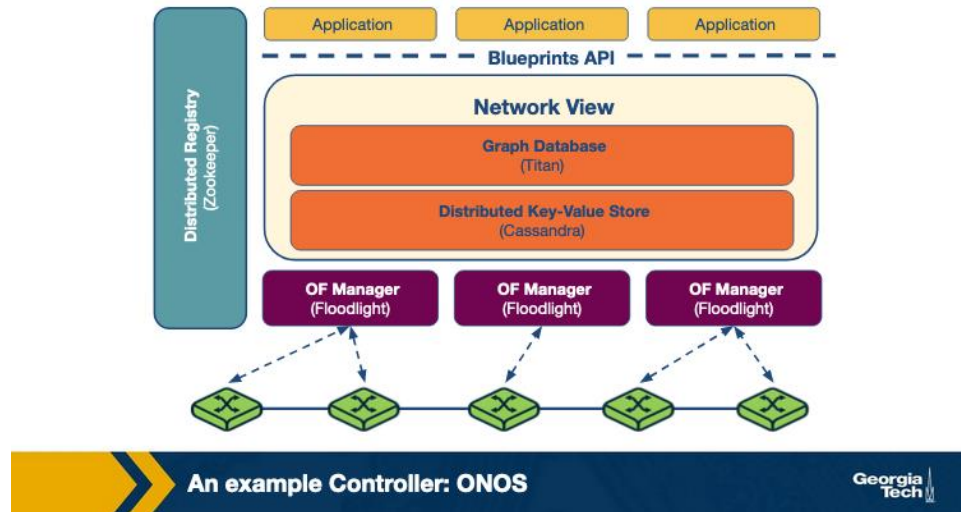


Figure: ONOS — distributed control with mastership and failover (Module 8)

- A. Each switch connects to exactly one ONOS instance and cannot fail over without manual operator intervention.
- B. Each switch connects to multiple ONOS instances; one acts as master, and on master failure a consensus election picks a new master.
- C. Each switch is its own controller; ONOS is only a telemetry collector for the operator.
- D. Switches connect to all instances simultaneously and any instance can write rules at will.

Correct answer: **B**

Why: ONOS: each switch has multiple instances + ONE master (consensus). One master writes at a time; remaining instances stand by and elect a new master if the current one fails.

Q30. [MCQ]

Scenario from the ONOS figure: an ONOS instance crashes while serving as master for several switches. What happens next?

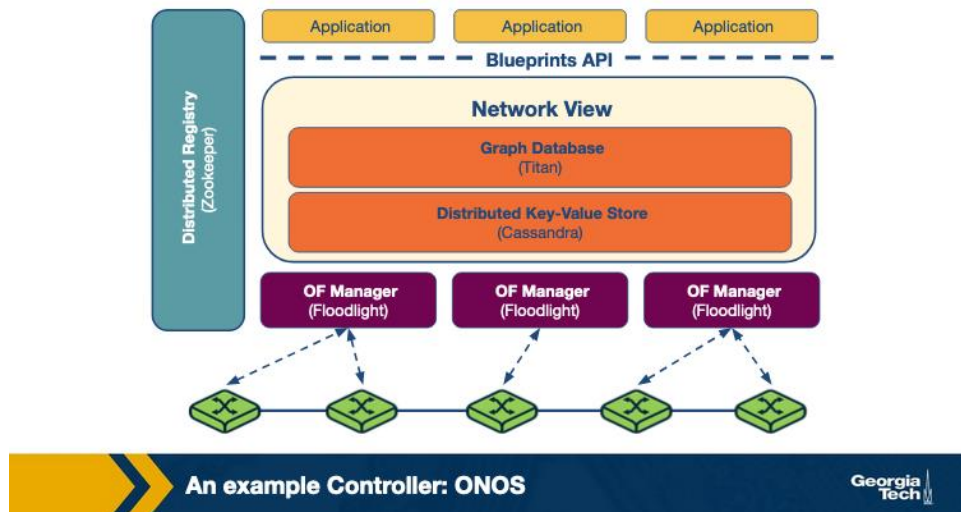


Figure: ONOS — mastership election after failure (Module 8)

- **A.** Switches automatically promote themselves to local-controller mode and re-converge with the wider AS later.
- **B.** All switches whose master died are stranded until the operator restarts the dead instance manually.
- **C.** The remaining connected instances run a consensus election (e.g., via ZooKeeper) and pick a new master for each affected switch.
- **D.** ONOS hands master responsibilities to the IXP's route server temporarily for those switches.

Correct answer: **C**

Why: Consensus election (ZooKeeper-style) picks new master per orphaned switch. When a master dies, the surviving instances run a Paxos/Raft-style election; affected switches gain a new master without operator intervention.

Programming the Data Plane: The Motivation

Q31. [MCQ]

What does the figure motivate that OpenFlow alone does NOT enable?

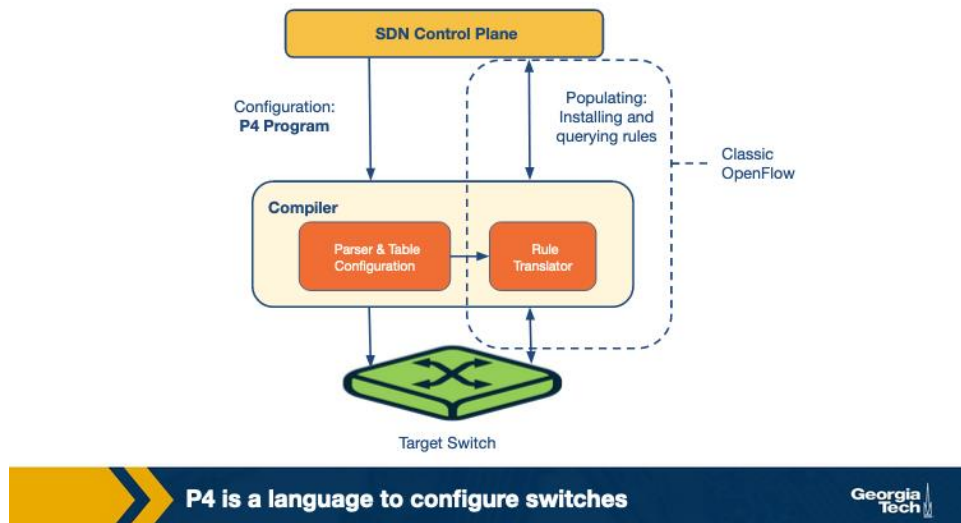


Figure: Motivation for programming the data plane (Module 8)

- A. Authentication of every packet at the network layer.
- B. Switching at line rate — the claim that OpenFlow switches cannot forward fast enough without P4 accelerating each match+action stage in hardware.
- C. Persistent connections between controllers and switches.
- D. Programmability of the switch itself — defining the parser and the set of match+action stages, not just filling in rules in a fixed pipeline.

Correct answer: **D**

Why: P4 programs the SWITCH ITSELF (parser + pipeline). OpenFlow only fills in rules in a fixed pipeline; P4 lets you DEFINE the pipeline — what headers exist, in what order they're parsed, what stages run.

Q32. [MCQ]

How does the module describe the relationship between P4 and OpenFlow?

- A. P4 configures the switch programmatically (parser + match+action tables); OpenFlow populates rules into a fixed-function pipeline — they are complementary.
- B. P4 is a replacement for all southbound APIs that eliminates the need for any controller, because each switch compiles and runs its own control logic independently.
- C. P4 is a control-plane routing protocol like BGP layered on top of OpenFlow.
- D. OpenFlow compiles down to P4 bytecode and runs only on legacy hardware.

Correct answer: **A**

Why: P4 (configure programmable pipeline) + OpenFlow (populate rules) are complementary. They live at different layers: P4 defines the data-plane program; OpenFlow (or its successors) installs runtime entries into the resulting tables.

Programming the Data Plane: P4's Forwarding Model

Q33. [MCQ]

The figure separates two classes of operations in P4. Which pair is correct?

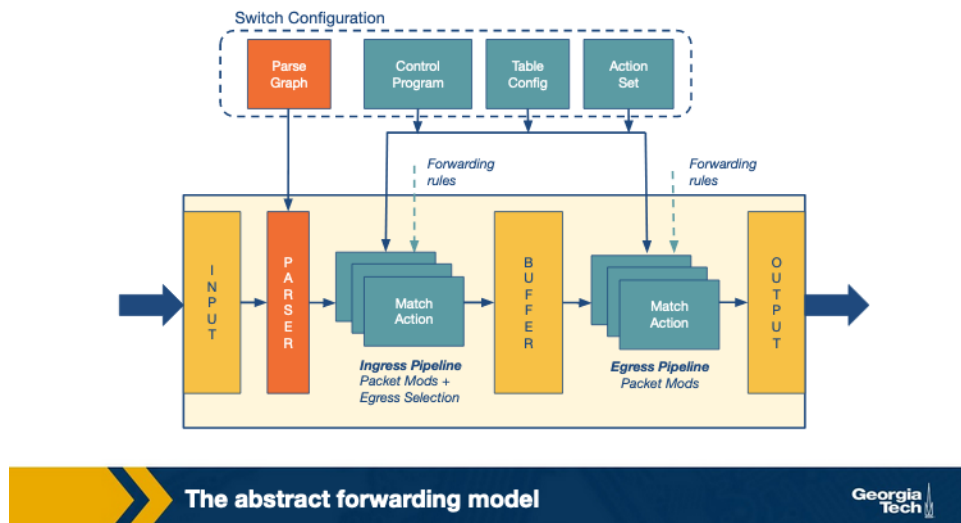


Figure: P4 forwarding model — configure vs populate (Module 8)

- A. Compile and Link, like a general-purpose language toolchain that builds an executable before loading it onto the switch.
- B. configure (program parser and order of match+action stages) and populate (add/delete entries in those tables).
- C. Encrypt and Decrypt, for line-rate IPsec.
- D. Advertise and Withdraw, like BGP UPDATE messages.

Correct answer: **B**

Why: Two ops in P4: CONFIGURE (program) + POPULATE (entries). Configure changes the parser/pipeline definition; populate fills entries into already-defined match+action tables.

Q34. [MCQ]

Which set of P4's design properties is identified by the lesson?

- A. Encryption, signing, and key rotation.
- B. Reliability, integrity, and confidentiality — the three security guarantees that P4 enforces in the data plane by signing and encrypting each packet at line rate.
- C. Reconfigurability (the switch can be reprogrammed), protocol independence (no fixed parser), and target independence (one program runs on different hardware targets).
- D. Compression, deduplication, and rate-limiting.

Correct answer: **C**

Why: P4 design properties: reconfigurable + protocol-independent + target-independent. The switch can be reprogrammed at runtime, the parser isn't tied to fixed protocols, and the same program compiles to multiple hardware targets.

Q35. [TF]

P4 lets the same control program describe how a switch parses packets AND how it processes them through match+action tables.

- True
- False

Correct answer: **True**

Why: P4 = parse + process in one program. The language describes both how to parse arriving packets (extracting headers) AND how match+action tables operate on those headers.

SDN Applications: Overview

Q36. [MCQ]

What is the role of SDN applications in the SDN architecture?

- **A.** They are the network-control programs that operators write on top of the northbound API to express policies (routing, security, traffic engineering, etc.).
- **B.** They are simply the data-plane match+action tables under a new name, compiled directly into switch silicon with no northbound API or controller involvement.
- **C.** They run inside the switches' silicon and have no controller involvement.
- **D.** They are end-user mobile apps that download bandwidth from the controller.

Correct answer: **A**

Why: SDN apps = network-control programs on northbound API. Routing, security, TE, monitoring — all expressed as programs that read controller state and install policy through the controller's northbound API.

SDN Application Example: A Software Defined Internet Exchange

Q37. [MCQ]

In the SDX architecture figure, each AS sees the illusion of its own virtual SDN switch connecting it to every other participant. Why is this abstraction useful?

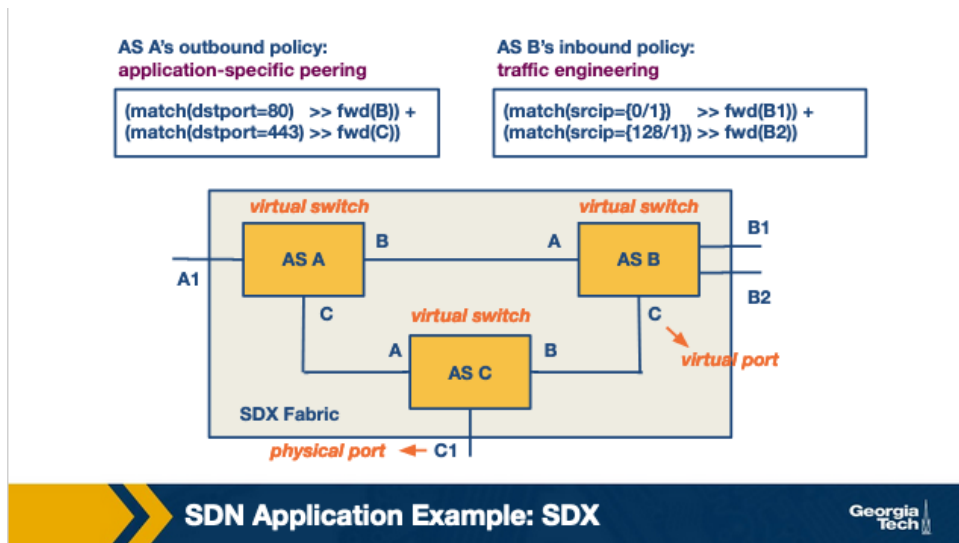


Figure: SDX — each AS sees a virtual SDN switch connecting it to all participants (Module 8)

- **A.** It eliminates BGP entirely from the Internet because each AS's virtual SDN switch replaces AS-level routing, so participants no longer exchange any route advertisements at the exchange point.
- **B.** Each AS can define its own forwarding policies (inbound and outbound) as if it were the sole participant; the SDX merges all participants' policies into a single physical-switch policy.
- **C.** Every AS sees the same policy as every other, removing per-AS customization.
- **D.** All participants share a single anycast IP for every advertised service.

Correct answer: **B**

Why: SDX virtual-switch abstraction: each AS writes policies as if alone. The SDX hides composition complexity: each participant reasons locally about its own policy, and the SDX merges all participants' policies into a coherent shared switch.

Q38. [MCQ]

Consider this Pyretic policy from the SDX section: `(match(dstport=80) >> fwd(B)) + (match(dstport=443) >> fwd(C))`. A packet arrives with destination port 22 (SSH). What does the parallel operator '+' do?

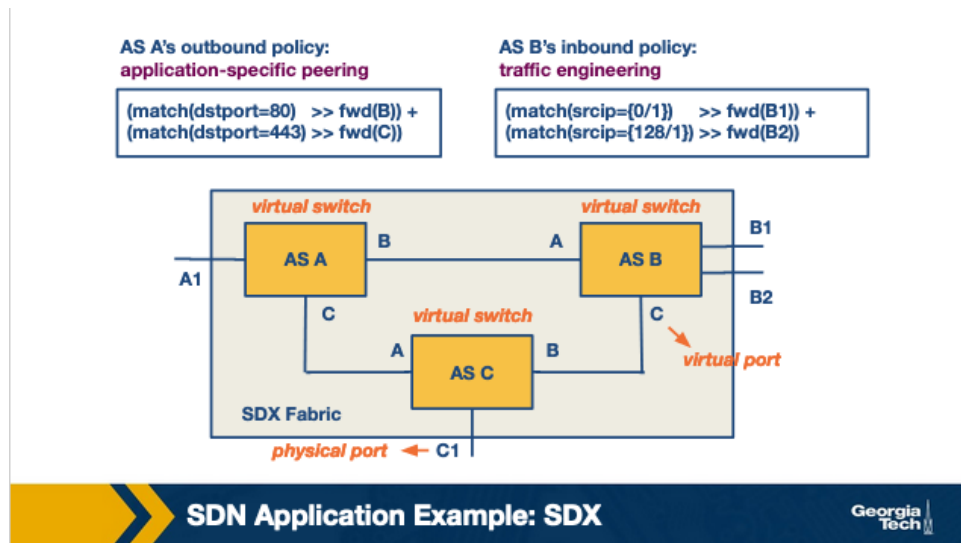


Figure: SDX with Pyretic policy (Module 8)

- **A.** The packet is forwarded to B because B is listed first.
- **B.** The packet is forwarded to BOTH B and C in parallel, since '+' duplicates every arriving packet to all listed forwarding actions.
- **C.** The packet is dropped — '+' returns the combined output of all sub-policies, and neither sub-policy matches port 22.
- **D.** The packet is sent to the controller via PacketIn for resolution.

Correct answer: **C**

Why: Pyretic '+' = parallel composition; no sub-policy matches => drop. The parallel operator combines outputs of multiple policies on the same packet; if no sub-policy matches, the combined output is empty so the packet is dropped.

Q39. [MCQ]

Scenario: an SDX participant AS A writes the policy 'match(dstport=80) >> fwd(B) + match(srcAS=AS_X) >> fwd(C)'. Another participant AS D writes 'match(dstport=80) >> fwd(E)'. How does the SDX combine these conflicting outbound policies for port 80?

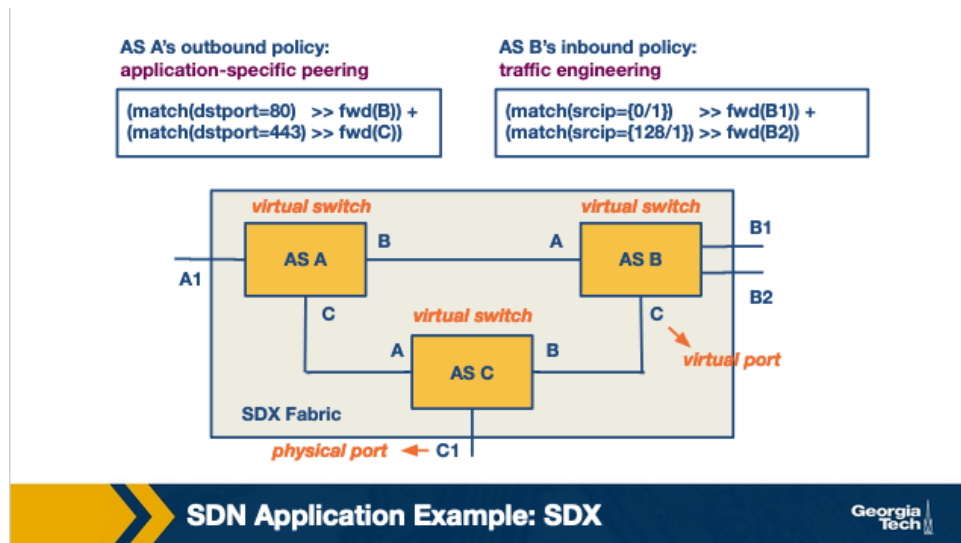


Figure: SDX policy composition (Module 8)

- **A.** SDX rejects both policies and forces the participants to use traditional BGP for port-80 traffic.
- **B.** The SDX drops every port-80 packet because A and D conflict on the destination port, and the exchange refuses to install two policies that match the same field until the participants manually resolve the collision through BGP.
- **C.** Only the first-installed policy wins; A or D loses depending on installation order.
- **D.** The SDX merges all participants' policies into one physical-switch policy, respecting per-participant ownership of each sub-policy — A's port-80 rule directs A's outbound flows, D's port-80 rule directs D's outbound flows.

Correct answer: **D**

Why: SDX composes per-participant policies into one master policy (per-participant ownership). Each participant's rules apply to its own traffic; the SDX's policy compiler merges them into a single switch policy while respecting ownership of sub-policies.

Q40. [TF]

SDX combines per-participant policies into ONE master policy installed on the shared switching fabric, while still letting each participant reason about its own policy locally.

- True
- False

Correct answer: **True**

Why: SDX merges participants' policies; each still reasons locally. The participant writes its policy as if alone; SDX handles composition and conflict resolution into the underlying physical switch fabric.

SDN Applications: Wide Area Traffic Delivery

Q41. [MCQ]

What is the main motivation for SDN-based wide-area traffic delivery (e.g., across a backbone)?

- **A.** Operators can centrally plan traffic engineering — directing flows across multiple backbone paths based on global utilization and policy — instead of relying on per-router heuristics.
- **B.** Wide-area links are too slow for distributed routing protocols to converge, so SDN is needed because only a central controller can run BGP fast enough across a long-haul backbone.
- **C.** SDN enables every backbone link to run at higher line rate than non-SDN equivalents.
- **D.** Wide-area SDN replaces BGP entirely on the public Internet.

Correct answer: **A**

Why: Central TE planner uses global utilization view. Traditional networks rely on per-router heuristics (OSPF weights); central TE can solve the whole-network optimization (LP) using a fresh global view of link loads.

Q42. [MCQ]

Scenario: an SDN traffic-engineering app sees that ONE backbone link is at 95% utilization while a parallel link is at 30%. What can it do that's HARDER in traditional networks?

- **A.** Install new OSPF link weights at every router simultaneously by physically replacing the line cards, which lets a traditional network shift load faster than any SDN controller could.
- **B.** Programmatically install rules to shift some flows to the under-utilized link based on a global view — without waiting for slow distributed routing protocols to react.
- **C.** Run BGP on every backbone router twice as fast.
- **D.** Pre-program every host on the network with a load-aware sender.

Correct answer: **B**

Why: Programmatically shift flows to under-utilized links — distributed protocols can't react fast enough. Centralized TE installs new flow rules immediately; OSPF/ISIS would have to re-converge through their own timers and bandwidth.

Q43. [TF]

Wide-area SDN deployments (e.g., Google's B4) demonstrated that centralized traffic engineering can sustain much HIGHER link utilization than traditional shortest-path routing.

- True
- False

Correct answer: **True**

Why: B4 demonstrated 90%+ link utilization via centralized TE. Google's WAN runs near full utilization by centrally planning flows across multiple paths — far above the 30-40% typical of shortest-path WANs.